

A completely UNIX project

ft_nm

Summary: This project is about recoding the command nm.

Version: 3.5

Contents

I	**Foreword**	**2**
II	**AI Instructions**	**3**
III	**Subjet**	**5**
IV	**Bonus Part**	**7**
V	**Submission and Peer Review**	**8**

Chapter I

Foreword

theorem 1 (Lagrange)**.** *For any finite group G, the order (number of elements) of every subgroup H of G divides the order of G.*

Proof. Let $\sim$ be the relation defined by: for all $x, y \in G$, $x \sim y$ if and only if there exists $a \in H$ such that $ax = y$. Let's show that $\sim$ is an equivalence relation.

reflexivity $1x = x$.

Symmetry if $ax = y$ then $x = a^{-1}y$.

transitivity If $ax = y$ and $by = z$, then $(ba)x = z$

The equivalence classes of $\sim$ form a partition of G. For $x \in G$, let $cl(x) = Hx$. If we show that all classes have the same cardinality, then we show that the cardinality of $cl(1) = H$ divides the cardinality of G.

Let $a, b \in G$. Let us define a bijection from Ha to Hb. Let $f : Ha \longrightarrow Hb$ as for all x in G, $f(x) = xa^{-1}b$. Let $g : Hb \longrightarrow Ha$ as for all x in G, $g(x) = xb^{-1}a$. For all $x \in G$, $f(g(x)) = xb^{-1}aa^{-1}b = x$ and $g(f(x)) = xa^{-1}bb^{-1}a = x$. So $g = f^{-1}$.

□

Chapter II

AI Instructions

🟢 Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

🟢 Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

🟢 Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter III

Subjet

Program Name	`ft_nm`
Files to Submit	
Makefile	`Yes`
Arguments	
External Function	`open(2) close(2) mmap(2) munmap(2) write(2) fstat(2) malloc(3) free(3) exit(3) perror(3) strerror(1) getpagesize(2)`
Libft authorized	
Description	`You have to recode the nm (with no options)`

To complete this project, you will have to use a Linux OS. You may use Docker or a virtual machine if needed. You have to work with ELF binaries. You have to handle x86_32 and x86_64 binaries, object files, and shared libraries (.so).

Use the `file` command to view details about a file. You can use the binaries located in your system (`/usr/bin/, /usr/lib/`...).



`man nm`

- This project will be corrected by humans only. You're allowed to organize and name your files as you see fit, but you must follow the following rules.
- The executable must be named `ft_nm`
- You must code in C and use a Makefile.
- Your Makefile must compile the project and contain the usual rules.
- If you choose to do so, you may use your own library for `ft_nm`. Also submit your `libft` folder including its own `Makefile` at the root of your repository. Your Makefile will have to compile the library, and then compile your project.
- Output is to be similar to nm on the symbols list (order, offset, padding...). Other aspects can have some small differences (filename, etc).

- Your program must behave like the system nm on every other aspect. You have to handle errors carefully. In no way can your program quit in an unexpected manner (Segmentation fault, bus error, double free, etc).
- Be cautious. There are many ways to lead your program out of the mapped content, may it be with non-null terminated string, incorrect offsets... Check everything.
- Make sure that every value that you parse is correct: for instance, when parsing flags, architecture, match your result with a reference to ensure its value is correct.

Chapter IV

Bonus Part



We will look at your bonus part only if your mandatory part is EXCELLENT. This means that you must complete the mandatory part, from beginning to end, your error management needs to be flawless, even in cases of twisted or improper usage. If that's not the case, your bonuses bonuses will be completely ignored.

You must manage the following options:

- -a
- -g
- -u
- -r
- -p

Chapter V

Submission and Peer Review

Submit your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Make sure to double-check the names of your folders and files to ensure they are correct.